Chapter 1, Data Transformation

These are the notes from working with the following raw datafiles:
Cluster Analysis Whole Images 01112016.xlsx
Cluster Analysis Whole Images 02172016.xlsx
Cluster Analysis 05302017.xlsx

The expermental data gathered by Juha Himanen in 2016 and 2017.
This workbook was last updated by Dave Cuthbert 2017-06-28.

The purpose to this notebook, 'Data Transformation', is to transform the data from the initial excel files into more usable formats. Analysis of the data is done in a 2d jupyter notebook, 'Data Analysis'

Imports and Initialization

Code Imports

In [35]:
```
import pandas as pd
import numpy as np
import collections
```

Import the raw data

For speed of loading, the original xlsx files were transformed to csv using LibreOffice.

In [36]:
```
df1 = None
df2 = None
df3 = None

df1 = pd.read_csv("2016-01-11.csv", low_memory=False)
df2 = pd.read_csv("2016-02-17.csv", low_memory=False)
df3 = pd.read_csv('2017-05-30.csv')
```

Combine the files and check the data counts.

The 2016 raw files have many rows that contain data taken at 2 different resolutions. After identifying the columns, this data can be tagged by resolution and then the combined rows from the raw file can be split into individual rows in a new dataframe for analysis. Create one dataframe for results using the 65 count threshold and another one for results using the 125 count threshold.

Note that the ordering of the reported results is reversed between the two files from 2016. T65 data is reported first in the 2016-01-11 file and T125 data is reported first in the 2016-02-17 file.

The 2017 file contains data collected without using any count threshold so there will be a greater number of observations for this data set than for the others.

Generate lists that combine the Threshold 65 data from coresponding columns in 01112016 and 02172016

In [37]:
```
image_name_65 = df1['ImageName '].tolist()
image_name_65.extend(df2['ImageName .1'].tolist())
time_point_65 = df1[' Time Point '].tolist()
time_point_65.extend(df2[' Time Point .1'].tolist())
roi_index_65 = df1[' ROI Index '].tolist()
roi_index_65.extend(df2[' ROI Index .1'].tolist())
indiv_cluster_pxls_65 = df1['Individual Cluster Area (pixels)'].tolist()
indiv_cluster_pxls_65.extend(df2['Individual Cluster Area (pixels).1'].tolis
t())
indiv_cluster_intensity_65 = df1['Individual Cluster Intensity'].tolist()
indiv_cluster_intensity_65.extend(df2['Individual Cluster Intensity.1'].toli
st())
num_clusters_ttl_65 = df1[' Number of Cluster (Total)'].tolist()
num_clusters_ttl_65.extend(df2[' Number of Cluster (Total).1'].tolist())
area_cluster_total_pxls_65 = df1[' Area of Protein Cluster (Total in pixels)
'].tolist()
area_cluster_total_pxls_65.extend(df2[' Area of Protein Cluster (Total in pi
xels).1'].tolist())
intensity_clusters_ttl_65 = df1[' Intensity of Protein Cluster  (Total)'].to
list()
intensity_clusters_ttl_65.extend(df2[' Intensity of Protein Cluster  (Total)
.1'].tolist())
```

Verify that the combined lists have the expected number of rows

In [38]:
```
if (len(df1['ImageName '].tolist()) + len(df2['ImageName .1'].tolist())) !=
len(image_name_65):
    print("Error: image_name_65")
if (len(df1[' Time Point '].tolist()) + len(df2[' Time Point .1'].tolist()))
 != len(time_point_65):
    print("Error: time_point_65")
if (len(df1[' ROI Index '].tolist()) + len(df2[' ROI Index .1'].tolist())) !
= len(roi_index_65):
    print("Error: roi_index_65")
if (len(df1['Individual Cluster Area (pixels)'].tolist()) + \
        len(df2['Individual Cluster Area (pixels).1'].tolist())) != \
        len(indiv_cluster_pxls_65):
    print("Error: indiv_cluster_pxls_65")
if (len(df1['Individual Cluster Intensity'].tolist()) + \
        len(df2['Individual Cluster Intensity.1'].tolist())) != \
        len(indiv_cluster_intensity_65):
    print("Error: indiv_cluster_intensity_65")
if (len(df1[' Number of Cluster (Total)'].tolist()) + \
        len(df2[' Number of Cluster (Total).1'].tolist())) != \
        len(num_clusters_ttl_65):
    print("Error: num_clusters_ttl_65")
if (len(df1[' Area of Protein Cluster (Total in pixels)'].tolist()) + \
        len(df2[' Area of Protein Cluster (Total in pixels).1'].tolist())) !
= \
        len(area_cluster_total_pxls_65):
    print("Error: area_cluster_total_pxls_65")
if (len(df1[' Intensity of Protein Cluster  (Total)'].tolist()) + \
        len(df2[' Intensity of Protein Cluster  (Total).1'].tolist())) != \
        len(intensity_clusters_ttl_65):
    print("Error: intensity_clusters_ttl_65")
```

Combine the lists for Threshold 65 into a dataframe and set the column order

In [39]:
```
df65 = None
df65 = pd.DataFrame(
   {'ImageName': image_name_65,
    'Time Point': time_point_65,
    'ROI Index': roi_index_65,
    'Individual Cluster Area': indiv_cluster_pxls_65,
    'Individual Cluster Intensity': indiv_cluster_intensity_65,
    'Total Number of Clusters': num_clusters_ttl_65,
    'Total Area of Protein Clusters': area_cluster_total_pxls_65,
    'Total Intensity of Protein Clusters': intensity_clusters_ttl_65
    })

df65 = df65[['ImageName',
    'Time Point',
    'ROI Index',
    'Individual Cluster Area',
    'Individual Cluster Intensity',
    'Total Number of Clusters',
    'Total Area of Protein Clusters',
    'Total Intensity of Protein Clusters']]
```

Generate lists that combine the Threshold 125 data from coresponding columns in 01112016 and 02172016

In [40]:
```
image_name_125 = df1['ImageName .1'].tolist()
image_name_125.extend(df2['ImageName '].tolist())
time_point_125 = df1[' Time Point .1'].tolist()
time_point_125.extend(df2[' Time Point '].tolist())
roi_index_125 = df1[' ROI Index .1'].tolist()
roi_index_125.extend(df2[' ROI Index '].tolist())
indiv_cluster_pxls_125 = df1['Individual Cluster Area (pixels).1'].tolist()
indiv_cluster_pxls_125.extend(df2['Individual Cluster Area (pixels)'].tolist
())
indiv_cluster_intensity_125 = df1['Individual Cluster Intensity.1'].tolist()
indiv_cluster_intensity_125.extend(df2['Individual Cluster Intensity'].tolis
t())
num_clusters_ttl_125 = df1[' Number of Clusters '].tolist()
num_clusters_ttl_125.extend(df2[' Number of Cluster (Total)'].tolist())
area_cluster_total_pxls_125 = df1[' Area of Protein Clusters (Total in Pixel
s)'].tolist()
area_cluster_total_pxls_125.extend(df2[' Area of Protein Cluster (Total in p
ixels)'].tolist())
intensity_clusters_ttl_125 = df1[' Intensity of Protein Clusters  (Total)'].
tolist()
intensity_clusters_ttl_125.extend(df2[' Intensity of Protein Cluster  (Total
)'].tolist())
```

Verify that the combined lists for Threshold 125 have the expected number of rows

In [41]:
```
if (len(df1['ImageName .1'].tolist()) + len(df2['ImageName '].tolist())) !=
len(image_name_125):
    print("Error: image_name_125")
if (len(df1[' Time Point .1'].tolist()) + len(df2[' Time Point '].tolist()))
 != len(time_point_125):
    print("Error: time_point_125")
if (len(df1[' ROI Index .1'].tolist()) + len(df2[' ROI Index '].tolist())) !
= len(roi_index_125):
    print("Error: roi_index_125")
if (len(df1['Individual Cluster Area (pixels).1'].tolist()) + \
        len(df2['Individual Cluster Area (pixels)'].tolist())) != \
        len(indiv_cluster_pxls_125):
    print("Error: indiv_cluster_pxls_125")
if (len(df1['Individual Cluster Intensity.1'].tolist()) + \
        len(df2['Individual Cluster Intensity'].tolist())) != \
        len(indiv_cluster_intensity_125):
    print("Error: indiv_cluster_intensity_125")
if (len(df1[' Number of Clusters '].tolist()) + \
        len(df2[' Number of Cluster (Total)'].tolist())) != \
        len(num_clusters_ttl_125):
    print("Error: num_clusters_ttl_125")
if (len(df1[' Area of Protein Clusters (Total in Pixels)'].tolist()) + \
        len(df2[' Area of Protein Cluster (Total in pixels)'].tolist())) !=
\
        len(area_cluster_total_pxls_125):
    print("Error: area_cluster_total_pxls_125")
if (len(df1[' Intensity of Protein Clusters  (Total)'].tolist()) + \
        len(df2[' Intensity of Protein Cluster  (Total)'].tolist())) != \
        len(intensity_clusters_ttl_125):
    print("Error: intensity_clusters_ttl_125")
```

Combine the lists for Threshold 125 into a dataframe and set the column order

In [42]:
```
df125 = None
df125 = pd.DataFrame(
   {'ImageName': image_name_125,
    'Time Point': time_point_125,
    'ROI Index': roi_index_125,
    'Individual Cluster Area': indiv_cluster_pxls_125,
    'Individual Cluster Intensity': indiv_cluster_intensity_125,
    'Total Number of Clusters': num_clusters_ttl_125,
    'Total Area of Protein Clusters': area_cluster_total_pxls_125,
    'Total Intensity of Protein Clusters': intensity_clusters_ttl_125
    })

df125 = df125[['ImageName',
    'Time Point',
    'ROI Index',
    'Individual Cluster Area',
    'Individual Cluster Intensity',
    'Total Number of Clusters',
    'Total Area of Protein Clusters',
    'Total Intensity of Protein Clusters']]
```

Remove the extra, hidden characters from the receptor names

In [43]:
```
df65['ImageName'].replace(to_replace=r'[^-_.a-zA-Z0-9]', value='', inplace=True, regex=True)
df125['ImageName'].replace(to_replace=r'[^-_.a-zA-Z0-9]', value='', inplace=True, regex=True)
```

Remove the empty rows from df125 that were created because some lines in the original files had T65 data but no T125 data

In [44]:
```
df125.drop(df125[df125['ImageName'].isnull()].index, inplace=True)
```

Pull the summary rows out of the main dataframes

In [45]:
```
totals65 = None
totals125 = None

totals65 = df65.loc[df65['Total Number of Clusters'].notnull()]
totals125 = df125.loc[df125['Total Number of Clusters'].notnull()]
```

Drop rows with no clusters reported

In [46]:
```
totals65 = totals65[totals65['Total Number of Clusters'] != 0]
```

In [47]:
```
totals125 = totals125[totals125['Total Number of Clusters'] != 0]
```

Fix the column type after deleting empty rows

In [48]:
```
totals65[['Total Intensity of Protein Clusters']] = totals65[['Total Intensity of Protein Clusters']].apply(pd.to_numeric)
```

In [49]:
```
totals125[['Total Intensity of Protein Clusters']] = totals125[['Total Inten
sity of Protein Clusters']].apply(pd.to_numeric)
```

Create observation tables

In [50]:
```
observations65 = None
observations125 = None

observations65 = df65.loc[df65['Total Number of Clusters'].isnull()]
observations125 = df125.loc[df125['Total Number of Clusters'].isnull()]
```

Drop the unneeded columns from the totals and main dataframes

In [51]:
```
for col_name in ['Total Number of Clusters', 'Total Area of Protein Clusters
', 'Total Intensity of Protein Clusters']:
    if col_name in observations65.columns:
        del observations65[col_name]
    if col_name in observations125.columns:
        del observations125[col_name]

for col_name in ['ROI Index', 'Individual Cluster Area','Individual Cluster
Intensity']:
    if col_name in totals65.columns:
        del totals65[col_name]
    if col_name in totals125.columns:
        del totals125[col_name]
```

Create helper columns with scaled scores

In [52]:
```
max_value = totals65['Total Number of Clusters'].max()
totals65.insert(loc=5, column='Total Number Scaled', value=round(((totals65[
'Total Number of Clusters'] / max_value) * 100), 2))
```

In [53]:
```
max_value = totals65['Total Area of Protein Clusters'].max()
totals65.insert(loc=5, column='Total Area Scaled',
                value=round(((totals65['Total Area of Protein Clusters'] / m
ax_value) * 100),
                            2))
```

In [54]:
```
max_value = totals65['Total Intensity of Protein Clusters'].max()
totals65.insert(loc=5, column='Total Intensity Scaled', value=round(((totals
65['Total Intensity of Protein Clusters'] / max_value) * 100), 2))
```

In [55]:
```
max_value = totals125['Total Number of Clusters'].max()
totals125.insert(loc=5, column='Total Number Scaled', value=round(((totals12
5['Total Number of Clusters'] / max_value) * 100), 2))
```

In [56]:
```
max_value = totals125['Total Area of Protein Clusters'].max()
totals125.insert(loc=5, column='Total Area Scaled', value=round(((totals125[
'Total Area of Protein Clusters'] / max_value) * 100), 2))
```

In [57]:
```
max_value = totals125['Total Intensity of Protein Clusters'].max()
totals125.insert(loc=5, column='Total Intensity Scaled', value=round(((total
s125['Total Intensity of Protein Clusters'] / max_value) * 100), 2))
```

Create a helper column to group results by receptor

First, get a list of prefixes

In [58]:
```
image_list65 = totals65['ImageName'].unique()
image_list65.sort()

prefixes = [i.split('-',1)[0] for i in image_list65]
prefixes = list(set(prefixes))
```

In [59]:
```
image_list125 = totals125['ImageName'].unique()
image_list125.sort()

prefixes = [i.split('-',1)[0] for i in image_list125]
prefixes = list(set(prefixes))
```

Then assign receptors to groups

In [60]:
```
receptor_types = collections.defaultdict(list)
for p in prefixes:
    for i in image_list65:
        if p in i:
            if receptor_types[p]:
                receptor_types[p].append(i)
            else:
                receptor_types[p] = [i]

for p in prefixes:
    totals65.loc[totals65['ImageName'].isin(receptor_types[p]), 'Receptor']
= p
```

In [61]:
```
receptor_types = collections.defaultdict(list)
for p in prefixes:
    for i in image_list125:
        if p in i:
            if receptor_types[p]:
                receptor_types[p].append(i)
            else:
                receptor_types[p] = [i]

for p in prefixes:
    totals125.loc[totals125['ImageName'].isin(receptor_types[p]), 'Receptor'
] = p
```

Create a helper column to group results by experiment step

In [62]:
```
steps = ['-l-pre', '-l-post', '-nl-pre', '-nl-post']
```

In [63]:
```
experiment_step = collections.defaultdict(list)
for s in steps:
    for i in image_list65:
        if s in i:
            if experiment_step[s]:
                experiment_step[s].append(i)
            else:
                experiment_step[s] = [i]

for s in steps:
    totals65.loc[totals65['ImageName'].isin(experiment_step[s]), 'Experiment
 Step'] = s
```

In [64]:
```
experiment_step = collections.defaultdict(list)
for s in steps:
    for i in image_list125:
        if s in i:
            if experiment_step[s]:
                experiment_step[s].append(i)
            else:
                experiment_step[s] = [i]


for s in steps:
    totals125.loc[totals125['ImageName'].isin(experiment_step[s]), 'Experime
nt Step'] = s
```

Create a helper column for run group

In [65]:
```
run_group = [('-1_', 1), ('-2_',2)]
```

In [66]:
```
for i in range(len(run_group)):
    totals65.loc[totals65['ImageName'].str.contains(run_group[i][0]), 'Run G
roup'] = run_group[i][1]
```

In [67]:
```
for i in range(len(run_group)):
    totals125.loc[totals125['ImageName'].str.contains(run_group[i][0]), 'Run
 Group'] = run_group[i][1]
```

Fix column types

In [68]:
```
totals65['Total Number of Clusters'] = totals65['Total Number of Clusters'].
apply(np.int)
totals125['Total Number of Clusters'] = totals125['Total Number of Clusters'
].apply(np.int)
```

In [69]:
```
totals65['Total Area of Protein Clusters'] = totals65['Total Area of Protein
 Clusters'].apply(np.int)
totals125['Total Area of Protein Clusters'] = totals125['Total Area of Prote
in Clusters'].apply(np.int)
```

In [70]:
```
totals65['Run Group'] = totals65['Run Group'].apply(np.int)
totals125['Run Group'] = totals125['Run Group'].apply(np.int)
```

Clean up the indices

In [71]:
```
totals65 = totals65.reset_index(drop=True)
totals125 = totals125.reset_index(drop=True)
```

Output the transformed dataframes so that they can be used later.

In [72]:
```
totals65.to_csv('T65_Totals.csv', index=False)
totals125.to_csv('T125_Totals.csv', index=False)
```